BEDROCK ARCHITECTURE

Bedrock Target Intrinsics

C Compiler Builtins and Header Interfaces

Draft

CONTENTS

LIST OF TABLES

1 SCOPE AND STATUS

This document defines the compiler-owned Bedrock C target interface: builtin names, implementation-reserved header wrappers, source signatures, lowering requirements, availability and privilege constraints, and compiler-visible effects. It is the contract shared by a Bedrock-aware C frontend, optimizer, backend, and the target headers.

Architectural instruction behavior remains defined by the Bedrock ISA Specification. The baseline calling convention is defined by the Bedrock C ABI. This document specifies how source programs reach target facilities; it does not duplicate their architectural or language semantics.

Interface name: `bedrock-target-intrinsics`

Source language: ISO C with implementation-reserved target interfaces

2 COMPILER INTERFACE MODEL

2.1 Names and Ownership

The compiler builtin namespace begins with `__builtin_bedrock_`. Each **Name** entry in this document denotes that prefix followed by the displayed name. For example, `cpuid` denotes `__builtin_bedrock_cpuid`. The ordinary header wrapper uses the `__bedrock_` prefix with the same name unless its entry specifies a macro spelling.

All of these identifiers are implementation-reserved. A builtin is a compiler primitive, not an external function symbol. The target headers own wrapper declarations and umbrella include topology; this document owns their ABI-visible meaning.

2.2 Header Topology and Exposure

`<bedrockintrin.h>` is the public umbrella. It includes the core, memory, integer, and floating-point families. `<bedrocksystemintrin.h>` is the system umbrella for system-register, cache, MMU, and processor-state facilities. Any family header may also be included directly.

Table 2-1. Target Intrinsic Header Families

Family	Header	Umbrella	Exposure
core	`<bedrockcoreintrin.h>`	public	unprivileged core operations
memory	`<bedrockmemoryintrin.h>`	public	user ordering and access hints
integer	`<bedrockintegerintrin.h>`	public	nonportable integer operations
floating point	`<bedrockfpuintrin.h>`	public	floating-point environment and classification
system registers	`<bedrocksysregintrin.h>`	system	architectural system-register access
cache	`<bedrockcacheintrin.h>`	system	cache management
MMU	`<bedrockmmuintrin.h>`	system	supervisor translation operations
processor state	`<bedrockstateintrin.h>`	system	runtime state management

2.3 Common Compilation Rules

Family wrappers are static inline functions unless an encoded immediate or type operand requires a macro. Every builtin listed by an enabled profile must be recognized by the compiler. If a required

optional feature is disabled and no fallback is specified here, compilation fails rather than emitting an unresolved call.

An encoded immediate must be an integer constant expression in the stated range. Header inclusion neither suppresses nor emulates architectural privilege checks. An operation is volatile only when its entry states an externally observable architectural effect. Compiler memory effects and hardware ordering are exactly those stated by the entry; one does not imply the other.

The **C interface** column uses `result(arguments)` shorthand. Exact declarations are in the named header; `u8`, `u16`, `u32`, and `u64` abbreviate the corresponding `uintN_t` types, and `result` denotes `__bedrock_query_result_t`. A clobber means a compiler memory clobber, while a named fence also has the hardware behavior defined by the ISA.

3 PUBLIC TARGET INTERFACE

3.1 Core Builtins

Header: `<bedrockcoreintrin.h>`

Table 3-1. Core Builtins

Name	C interface	Lowering	Availability, constraint, and effect
`cpuid`	`u64(u64 selector)`	`CPUID`	User allowed; reads discovery state and not C memory.
`read_status`	`uint16_t(void)`	`RDSTATUS`	Unprivileged; reads `STATUS` and not C memory.
`rdpmc`	`uint64_t(counter_id)`	`RDPMC`	Unprivileged; ID is a constant in 0 through 65535; zero and unavailable IDs fault with `INVALID_SELECTOR`.
`breakpoint`	`void(void)`	`BKPT`	Any privilege; always raises the architectural breakpoint/debug exception.
`trace`	`void(marker)`	`TRACE`	Unprivileged; marker is a constant in 0 through 65535; emits a marker when enabled and otherwise acts as a NOP.
`yield`	`void(void)`	`YIELD`	Any privilege; scheduling hint, not a compiler or memory barrier.
`wait`	`void(void)`	`WAIT`	Unprivileged PAUSE-like convergent hint; retires normally and permits a finite implementation-selected delay, including zero, before subsequent execution.

3.2 Memory Builtins

Header: `<bedrockmemoryintrin.h>`

Table 3-2. Memory Builtins

Name	C interface	Lowering	Constraint and effect
`read_fence`	`void(void)`	`RFENCE`	Compiler and hardware read-ordering barrier.
`write_fence`	`void(void)`	`WFENCE`	Compiler and hardware write-ordering barrier.

Name	C interface	Lowering	Constraint and effect
address_fence	void(void)	AFENCE	Full compiler memory clobber and hardware address fence.
nontemporal_store_u8	void(u8 *,u8)	MOVNT.B	One writable byte; one hinted store, neither volatile nor a fence.
nontemporal_store_u16	void(u16 *,u16)	MOVNT.W	One writable 16-bit object; one hinted store, neither volatile nor a fence.
nontemporal_store_u32	void(u32 *,u32)	MOVNT.L	One writable 32-bit object; one hinted store, neither volatile nor a fence.
nontemporal_store_u64	void(u64 *,u64)	MOVNT.Q	One writable 64-bit object; one hinted store, neither volatile nor a fence.

3.3 Integer Builtins

Header: <bedrockintegerintrin.h>

Table 3-3. Integer Builtins

Name	C interface	Lowering	Result and effect
clmul_u8	u8(u8,u8)	CLMUL.B	Pure; low 8 bits of the carryless product.
clmul_u16	u16(u16,u16)	CLMUL.W	Pure; low 16 bits of the carryless product.
clmul_u32	u32(u32,u32)	CLMUL.L	Pure; low 32 bits of the carryless product.
clmul_u64	u64(u64,u64)	CLMUL.Q	Pure; low 64 bits of the carryless product.

3.4 Floating-Point Builtins

Header: <bedrockfpuintrin.h>

Table 3-4. Floating-Point Builtins

Name	C interface	Lowering	Availability, constraint, and effect
read_fstatus	uint16_t(void)	RDFSTATUS	Unprivileged; access to the floating-point environment.
write_fstatus	void(uint16_t)	WRFSTATUS	Unprivileged; reserved bits follow the ISA; changes the floating-point environment.
read_fflags	uint16_t(void)	RDFFLAGS	Unprivileged; reads accrued floating-point exception flags.
write_fflags	void(uint16_t)	WRFFLAGS	Unprivileged; reserved bits follow the ISA; writes accrued flags.
fclass_f32	uint16_t(float)	FCLASS.S	FPU required; pure ten-class one-hot classification; does not update FFLAGS.
fclass_f64	uint16_t(double)	FCLASS.D	FPU required; pure ten-class one-hot classification; does not update FFLAGS.

3.5 Approximate Transcendental Lowering Policy

A strict language-library operation shall not be lowered to an FPTRANSA instruction. A compiler may select FPTRANSA only when the source uses a separately defined approximate interface or an active compiler mode permits approximate mathematics. The selected instruction remains subject to its ISA accuracy, special-value, and floating-point exception-condition contract.

4 SYSTEM TARGET INTERFACE

The compiler emits the requested architectural operation; runtime privilege and policy checks remain active.

4.1 System-Register Builtins

Header: `<bedrocksysregintrin.h>`

Table 4-1. System-Register Builtins

Name	C interface	Lowering	Availability, constraint, and effect
`write_status`	`void(uint16_t)`	`WRSTATUS`	Supervisor; reserved and privilege-controlled bits follow the ISA; compiler-visible side effect.
`read_control_register`	`uint64_t(selector)`	`RDCR`	Supervisor; selector is a constant in 0 through 65535 naming a defined register.
`write_control_register`	`void(selector,u64)`	`WRCR`	Supervisor; defined constant selector required; writes the register and clobbers memory.
`read_segment_register`	`uint64_t(selector)`	`RDSEG`	Unprivileged; selector is a compile-time `__bedrock_segment_register_t`.
`read_code_segment`	`uint64_t(void)`	`RDSEG CS`	Unprivileged; reads the current CS image through the dedicated source-only encoding.
`write_segment_register`	`void(selector,image)`	`WRSEG`	Unprivileged; validates the SREG image, changes address interpretation, and fully clobbers memory.

The system-register header defines `__bedrock_control_register_t` constants for every published control-register selector. The constants are names for the architectural selector values; they do not weaken the privilege, reserved-bit, or per-register validation performed by RDCR and WRCR.

It also defines the pure value macros `__BEDROCK_SEGMENT_IMAGE`, `__BEDROCK_SEGMENT_IMAGE_FOR_BASE`, and `__BEDROCK_SEGMENT_DISABLED`. They construct 64-bit segment images without reading or writing architectural state.

`__BEDROCK_SEGMENT_IMAGE(base_page,exponent,mantissa, bounds_only)` places the low 52, 5, and 6 bits of the first three operands in their architectural fields and normalizes `bounds_only` to zero or one. Each operand is evaluated once. A valid enabled image uses an exponent from 0 through 31 and a mantissa from 1 through 63. The macro only packs fields; it does not validate the resulting base, span, limit, or address canonicality.

`__BEDROCK_SEGMENT_IMAGE_FOR_BASE(base,...)` encodes $\lfloor$`(uint64_t)base`$/4096\rfloor$ in the base-page field. It discards the low 12 bits, so a misaligned operand names the containing page and does not preserve the original byte address. The macro imposes no alignment precondition and does not adjust a later offset to compensate for the discarded bits. `__BEDROCK_SEGMENT_DISABLED` is the canonical all-zero disabled image.

4.2 Cache-Management Builtins

Header: `<bedrockcacheintrin.h>`

All cache range operations evaluate the address once and obtain the maintenance granule from CPUID `CACHE_TOPOLOGY`. A zero length emits no cache-maintenance instruction. A nonzero, non-wrapping byte range selects every maintenance block that intersects the range and lowers to one block instruction per selected block. Each instruction translates its own block address. If a later instruction faults, blocks completed by earlier instructions remain completed.

For a nonzero length, the mathematical half-open range $[\mathrm{address}, \mathrm{address} + \mathrm{length})$ shall lie within the 64-bit address space. Otherwise the behavior of the call is undefined and no architectural cache-maintenance sequence is required. An implementation may diagnose a provably wrapping constant range. A valid range end is calculated mathematically, not modulo 2^{64}.

Table 4-2. Cache-Management Builtins

Name	C interface	Lowering	Availability and effect
`flush_dcache`	`void(void *,size_t)`	`FLSHDCACHE`	Supervisor; applies FLSHDCACHE to every CPUID maintenance block intersecting the range and fully clobbers memory.
`invalidate_dcache`	`void(void *,size_t)`	`INVDCACHE`	Supervisor; applies INVDCACHE to every CPUID maintenance block intersecting the range and fully clobbers memory.
`invalidate_icache`	`void(void *,size_t)`	`INVICACHE`	Supervisor; applies local INVICACHE to every CPUID maintenance block intersecting the range, serializes instruction fetch, and fully clobbers memory.
`writeback_dcache`	`void(void *,size_t)`	`WRBKDCACHE`	Supervisor; applies WRBKDCACHE to every CPUID maintenance block intersecting the range and fully clobbers memory.
`sync_cache`	`void(void *,size_t)`	`SYNCCACHE`	Supervisor; applies SYNCCACHE to every CPUID maintenance block intersecting the range, serializes instruction fetch, and fully clobbers memory.

4.3 MMU Builtins

Header: `<bedrockmmuintrin.h>`

Table 4-3. MMU Builtins

Name	C interface	Lowering	Constraint and effect
`invalidate_tlb`	`void(void)`	`INVTLB`	Supervisor; translation-state side effect and full memory clobber.
`invalidate_page`	`void(const void *)`	`INVPAGE`	Supervisor; argument supplies a linear address and invalidates any matching cached leaf mapping containing it; the operation fully clobbers memory.

Name	C interface	Lowering	Constraint and effect
`invalidate_asid`	`void(asid)`	`INVASID`	Supervisor; ASID is a constant in 0 through 65535 and the operation fully clobbers memory.
`switch_page_table`	`void(uint64_t ptcr)`	`SWPT`	Supervisor; valid page-table image required; changes address space and fully clobbers memory.
`switch_page_table_asid`	`void(uint64_t,uint16_t)`	`SWPTA`	Supervisor; valid image and low 16-bit ASID; changes address space and fully clobbers memory.
`virtual_to_physical`	`result(address)`	`VTOP`	Supervisor; wrapper returns value and operation-defined `FLAGS` captured atomically; page walk without access to the translated C object.
`page_table_query`	`result(level,address)`	`PTQUERY`	Supervisor; level is an integer constant expression in 1 through 5; wrapper atomically returns value and `FLAGS`; page walk without access to the queried C object.

4.4 Processor-State Builtins

Header: `<bedrockstateintrin.h>`

Table 4-4. Processor-State Builtins

Name	C interface	Lowering	Constraint and effect
`save_processor_state`	`void(void *area)`	`SAVE`	Writable, CPUID-sized, 4096-byte-aligned area; writes architectural state and fully clobbers memory.
`restore_processor_state`	`void(const void *area)`	`RESTORE`	Readable, feature-valid, 4096-byte-aligned area; reads the complete selected processor-state image; may define `R0--R15`, `FLAGS`, valid `GS` registers, and extension state; fully clobbers memory and prevents TLS or segmented memory accesses from moving across a restored `GS` addressing context.

Both operations are unprivileged, but restoration of `STATUS` remains privilege-restricted. The backend must model every restored register and the full memory and addressing-context clobber explicitly. It must not treat restore as an opaque call or as only a memory effect.

5 SHARED HEADER TYPES

Table 5-1. Target Intrinsic Shared Types

Type	ABI contract
`__bedrock_segment_register_t`	selectors for `DS`, `SS`, and `GS0` through `GS5`; `CS` instead uses the fixed `RDSEG CS` source encoding
`__bedrock_control_register_t`	constants for the architectural RDCR and WRCR selector namespace
`__bedrock_query_result_t`	`u64 value; u16 flags; u16 reserved[3]`